

ReservalQ

Priorización de reservas hoteleras con aprendizaje automático

Microproyecto 3 · Computación en la Nube · UAO
Prof. Oscar Mondragón

Equipo

Juan Ospina Tenorio
Natalia Hernández Piedrahita
Miguel Ángel Diuza

Problema de negocio

Hotel Brisa del Valle es un cliente ficticio de Cali. Su equipo de reservas dispone de tiempo limitado para revisar confirmaciones. ReservalQ ordena las reservas según un índice asociado a cancelación y selecciona una lista de tamaño K, ajustada a esa capacidad. El personal decide qué acciones realizar.

Resultado retrospectivo	Valor
Reservas reservadas para prueba	7.990
Precisión del 20 % prioritario	44,1 %
Cancelaciones capturadas en ese 20 %	38,5 %
Concentración frente a selección aleatoria esperada	1,93 veces

Los datos son reales e históricos, de dos hoteles de Portugal. No pertenecen al cliente ficticio. Estas métricas no prueban ahorros, cancelaciones evitadas ni generalización a Colombia.

Estado de la evidencia

Entrenamiento, evaluación, inferencia y trece pruebas verificados localmente. Componentes de Azure ML configurados. Esta revisión no contiene un trabajo remoto completado ni capturas verificadas de navegación web. La configuración por sí sola no demuestra ejecución en Azure.

1. Requerimientos y alternativas

La necesidad se traduce en una decisión verificable: qué reservas revisar primero cuando existe una capacidad K. La alerta por umbral es un análisis complementario y no obliga a contactar a todos los registros señalados.

ID	Requerimiento	Criterio de aceptación
R1	Analizar una reserva	Validar diez variables y devolver índice, decisión y versión.
R2	Priorizar revisión	Seleccionar exactamente K registros y exportar la lista.
R3	Analizar un lote	CSV con 1–500 filas, máximo 150 KB y el mismo contrato.
R4	Evaluar sin usar el futuro	Particiones temporales disjuntas y madurez de etiquetas.
R5	Comparar y explicar	Candidatos, métricas, matriz y ejemplos de aciertos y errores.
R6	Ejecutar en Azure ML	Tres componentes, trabajo Completed y artefactos verificables.
R7	Limitar consumo	CPU, mínimo cero, máximo un nodo y límites de duración.

Alternativas consideradas

Alternativa	Ventaja	Limitación
Reglas manuales	Simplicidad y explicación	Umbrales rígidos y mantenimiento manual.
Clasificador y lista	Comparación y prioridad medible	Necesita datos y seguimiento de errores.
AutoML	Exploración automatizada	Más ensayos y consumo variable.

Se elige clasificación supervisada con revisión humana. No se requiere un chatbot para resolver esta decisión tabular. No hay integración con un sistema hotelero productivo, envío de mensajes, modificación de reservas ni cobros automáticos.

2. Datos, alcance y variables

Antonio, Almeida y Nunes publicaron 119.390 reservas de dos hoteles portugueses con llegadas en 2015–2017. Se conserva la distribución de TidyTuesday y su huella SHA256. Licencia de los datos originales: CC BY 4.0. [1, 2]

Tratamiento	Registros
Archivo original	119.390
Alcance antes de deduplicar	55.053
Duplicados exactos retirados	7.603
Después de deduplicar	47.450
Incluidos en las tres particiones	35.387
Fuera de períodos o madurez	12.063

Contrato de diez variables

Numéricas: lead_time, arrival_month, stays_in_weekend_nights y stays_in_week_nights.

Categorías: hotel, meal, market_segment, distribution_channel, reserved_room_type y customer_type.

Se admiten reservas con 0–60 días de anticipación, 1–30 noches totales, mes 1–12 y categorías conocidas. El diccionario completo está en ejemplos-csv/LEEME.md. No se usan identificadores de huéspedes ni contactos.

Exclusiones para reducir fuga de información

Estado final, fecha de estado, habitación asignada y cambios pueden reflejar hechos posteriores. ADR y depósitos no garantizan disponibilidad al crear la reserva. Se excluyen también país e identificadores de agencia o empresa.

Limitaciones del origen

La fecha de creación se deriva de llegada menos anticipación. No hay versiones históricas de los campos. La deduplicación puede eliminar reservas legítimas indistinguibles y no se puede excluir toda dependencia entre huéspedes recurrentes o grupos. El alcance no representa todos los hoteles ni todos los horizontes de reserva.

3. Método y selección

Partición	Creación de reserva	Resultado antes de	n / cancelaciones
Entrenamiento	Jul 2015–jun 2016	01/09/2016	21.236 / 3.933
Validación	Sep–nov 2016	01/02/2017	6.161 / 1.346
Prueba	Feb–may 2017	01/09/2017	7.990 / 1.831

Los espacios entre períodos permiten que los desenlaces anteriores sean conocidos antes de la evaluación siguiente. Las particiones no se mezclan aleatoriamente. El manifiesto conserva los límites exactos y la prueba comprueba que no se comparten registros.

Protocolo reproducible

1. Fijar alcance, fechas y semilla 42.
2. Ajustar escalado, codificación y modelos solo en entrenamiento.
3. Elegir el mayor average precision en validación.
4. Fijar umbral por máximo F1 en una cuadrícula de 0,05 a 0,95.
5. Evaluar la prueba reservada y conservar todas las predicciones.

Modelo	AP validación	ROC AUC validación
Base de referencia	0.2185	0.5000
Regresión logística	0.4027	0.7134
Random Forest	0.3677	0.6714
Gradient Boosting	0.4103	0.7105

Modelo elegido

Gradient Boosting, con umbral 0,17. La diferencia frente a regresión logística es modesta; no se afirma superioridad universal. Las configuraciones se limitan a una comparación manejable y no se ajustan usando la prueba.

Se fijan versiones de Python y librerías. `selection.json` documenta candidatos, barrido de umbral, duración de entrenamiento y entorno. `model.joblib` conserva transformaciones y clasificador juntos para evitar diferencias entre preparación e inferencia.

4. Arquitectura y flujo



Diseño propuesto para Azure ML; ejecución local verificada, Azure pendiente.

Componente	Responsabilidad
Workspace y Blob	Organizar trabajos, conservar entradas y artefactos.
Clúster CPU DS2 v2	Ejecutar con mínimo 0, máximo 1 nodo e inactividad de 120 s.
Preparación	Aplicar alcance, deduplicación, fechas y particiones.
Entrenamiento	Comparar candidatos y fijar modelo y umbral.
Evaluación	Calcular métricas sobre prueba y exportar evidencia.
Registro y descarga	Versionar el modelo y comprobar su huella.
Aplicación local	Inferencia individual, lotes y lista por capacidad.

Los componentes personalizados CLI v2 reutilizan pipeline.py. La entrada raw y las salidas splits, trained y report definen sus relaciones. Los componentes clásicos y personalizados no se mezclan en el mismo pipeline. [3]

El modelo descargado evita mantener un endpoint de inferencia. artifacts/runtime.json vincula la aplicación con trabajo, estado Completed, versión y SHA256. La etiqueta de origen Azure exige coincidencia con el archivo cargado. La ejecución remota de esta revisión sigue sin evidencia de finalización.

5. Resultados y decisión

Métrica de prueba	Resultado
Average precision	0,4134
ROC AUC	0,7307
Detección al umbral 0,17	80,1 %
Precisión al umbral 0,17	35,1 %
F1	0,4879
Brier del índice sin calibrar	0,1560

Desenlace real	Con alerta	Sin alerta
Canceló	1.467	364
No canceló	2.715	3.444

El umbral captura muchas cancelaciones y genera 2.715 falsas alertas. El hotel no tiene que revisar automáticamente todas las alertas: puede definir K según su capacidad.

Revisar el 20 % prioritario

Se seleccionan 1.598 registros y 705 cancelaron. La precisión es 44,1 % y se captura 38,5 % de las cancelaciones. Frente a la frecuencia base de 22,9 %, la concentración es 1,93 veces la esperada con una selección aleatoria. Es una comparación retrospectiva, no un experimento de intervención.

Explicación y límites

La importancia por permutación utiliza 2.500 reservas de validación y tres repeticiones. Describe sensibilidad global, no causalidad individual. El intervalo Wilson de detección es aproximado y no corrige dependencia entre reservas. Los índices no son probabilidades calibradas.

6. Implementación y comprobación

El servidor local carga el modelo y escucha en 127.0.0.1. La interfaz tiene cuatro vistas: decisiones, reserva individual, evidencia y diseño. El usuario puede variar K, editar una reserva, consultar casos de error, cargar un lote y exportar los resultados.

Ruta	Comportamiento
GET /api/summary	Métricas, ejemplos y procedencia comprobada.
GET /api/health	Estado del servicio, modelo y huella.
GET /api/sample.csv	Ocho reservas listas para cargar.
POST /api/predict	Una reserva validada y resultado real del modelo.
POST /api/batch	Entre 1 y 500 reservas; mismo contrato.

Ejecutar y cargar un lote

```
python3.12 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
python app.py --port 8765
```

Abrir <http://127.0.0.1:8765>. En la vista de reserva, seleccionar ejemplos-csv/reservas-listas.csv y analizar. El archivo contiene exactamente las diez columnas; no incluye el desenlace. El resultado se descarga como JSON.

Trece pruebas verificadas

Integridad del archivo; separación temporal; madurez de etiquetas; exclusión de desenlaces; selección en validación; concordancia de métricas; modelo guardado; rechazo de valores inválidos; categorías; API individual y por lote; solicitudes incorrectas; origen de peticiones; procedencia y CSV. Las comprobaciones están agrupadas en trece métodos de prueba.

```
python -m unittest discover -s tests -v
```

GitHub Actions ejecuta estas pruebas para cada cambio. Las pruebas del servidor no sustituyen una revisión visual de navegación. La captura de pantallas está bloqueada en esta revisión por la política de la herramienta de navegador.

7. Costos, evidencia y conclusiones

Concepto	Supuesto	USD
CPU DS2 v2	1 hora × 0,146	0,146
Servicios auxiliares	Reserva supuesta de práctica breve	0,500
Margen	Imprevistos del escenario	0,354
Total presupuestado	Una ejecución acotada	1,000

Referencia Linux East US consultada el 25/09/2026 UTC. La reserva auxiliar no es una cotización de todos los servicios y puede ser insuficiente si se conservan durante más tiempo. El total es un escenario, no factura ni bloqueo automático del gasto. No se registra consumo real de Azure ML en esta revisión. [4, 5]

Controles y evidencia de aceptación

Un nodo como máximo, mínimo cero, 120 segundos de inactividad y límites por etapa. Después de un trabajo se deben conservar salidas, verificar nodos liberados y consultar costos con fecha. Storage y Container Registry pueden seguir cobrando; las alertas de presupuesto no detienen el gasto.

La aceptación en Azure exige trabajo Completed, etapas, entorno resuelto, modelo registrado, salidas descargadas y huella coincidente con la aplicación. El repositorio conserva la configuración y las evidencias locales; los estados pendientes se explican en docs/EVIDENCIAS.md.

Conclusión

El experimento demuestra una priorización histórica con capacidad limitada y expone sus errores. Para utilizarlo en el hotel se requieren datos locales, auditoría de disponibilidad temporal de las variables y un experimento que mida el efecto de las acciones. La evidencia disponible no demuestra beneficios financieros.

Fuentes

1. Antonio, Almeida y Nunes: datos hoteleros
2. TidyTuesday: CSV y diccionario
3. Microsoft: componentes de Azure ML
4. Microsoft: API de precios
5. Microsoft: control de costos